ANSIBLE

ANSIBLE – DEV ENVIRONMENT

Provision and Configure Ansible Server (Control Node)

1. Setup EC2 instance
2. Setup hostname
3. Create ansadmin user
4. Add user to sudoers file
5. Generate ssh keys
6. Enable password-based login
7. Install ansible

Provision and Configure Target Servers (Managed Nodes)

1. Setup EC2 instance
2. Setup hostname
3. Create ansadmin user
4. Add user to sudoers file
5. Generate ssh keys
6. Enable password-based login
7. Install ansible

ANSIBLE - CONFIGURATION MANAGEMENT

What is Ansible?

Ansible is an open source, command-line IT automation software application written in Python. It can configure systems, deploy software, and orchestrate advanced workflows to support application deployment, system updates, networking configuration and operation, and more.

https://www.redhat.com/en/topics/automation/ansible-vs-salt

What is Configuration Management?

Configuration management refers to the practice of maintaining a system's desired state by **automating** the setup, deployment, and ongoing management of IT infrastructure. In **Ansible**, configuration management is achieved through **idempotent playbooks** that define the intended state of servers, applications, and services.

SIMPLE ANSIBLE PLAYBOOK

This **playbook** ensures that an Apache Server is:

1. Installed
2. Enabled
3. Running

Explanation

1. Ensures Apache (httpd) is installed **(state: present**).
2. Ensures the Apache service is running and enabled at boot (**state: started, enabled: yes**).
3. Running this playbook, multiple times won't reinstall or restart Apache unnecessarily **(idempotency).**

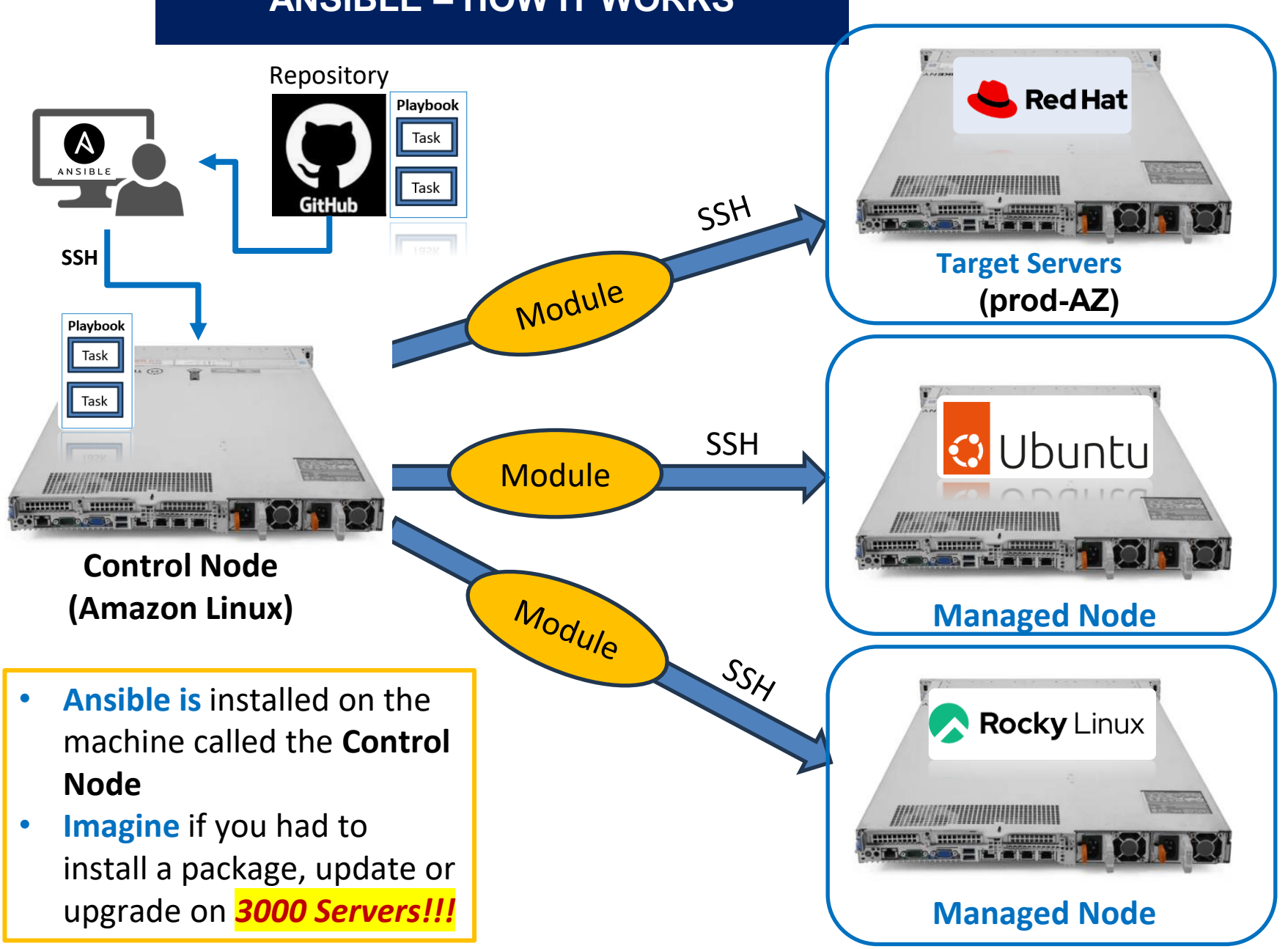
```
---
- name: Configure Apache Web Server
  hosts: web_servers
  become: yes  # Run tasks as root

  tasks: TASKS
    - name: Install Apache
      ansible.builtin.yum:
        name: httpd
        state: present

    - name: Start and Enable Apache Service
      ansible.builtin.service:
        name: httpd
        state: started
        enabled: yes
```

https://docs.ansible.com/ansible/9/collections/ansible/builtin/yum_module.html

ANSIBLE – HOW IT WORKS



- **Ansible is** installed on the machine called the **Control Node**
- **Imagine** if you had to install a package, update or upgrade on ***3000 Servers!!!***

TYPES OF SERVERS IN THE INDUSTRY

The list below gives an idea of servers and what they are used for in different companies:

Server = Managed Node = Targer Servers = Inventory (of Hosts)

1. Development Server (Dev Servers)
2. Performance Testing Servers (PT Servers)
3. Universal Testing Servers (UAT Servers)
4. System Integration Servers (SIT Servers)
5. Production Servers (PROD Servers)

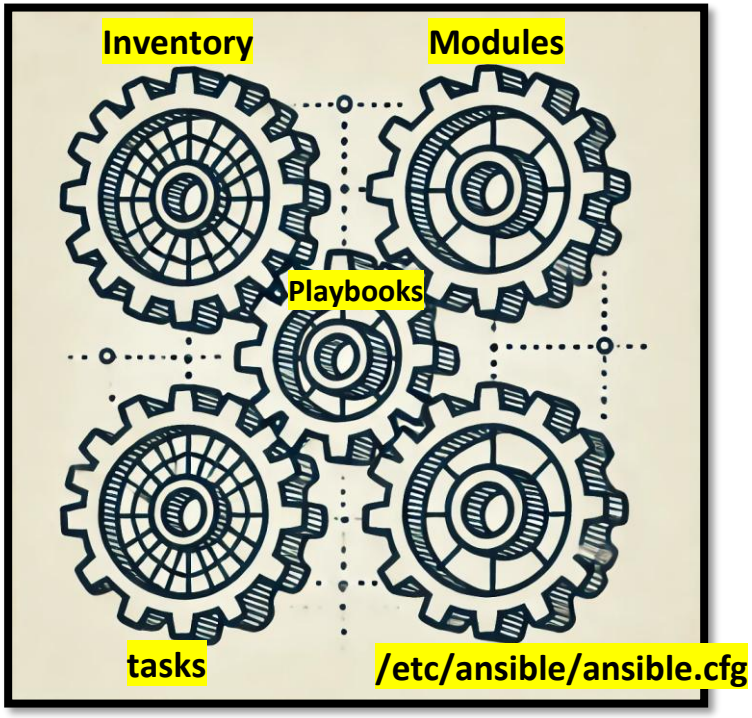
- Database Servers (DB Servers)
- Webservers
- Cache Servers
- Load Balancer Servers
- CI / CD Servers

ANSIBLE - COMPONENTS

What are Ansible Components:

There are five (5) basic components of Ansible

1. **Playbooks –** text file(s) that contain the play as .yaml file on the Control Node
2. **Inventory / hosts –** Target Servers defined in configuration files
3. **Modules – standard standalone mini code or** “tools” to make changes on target (managed nodes)
4. **Tasks –** task assigned in a playbook is carried out by a module
5. Ansible **Configuration File**



KEY ASPECTS OF CONFIG MANAGEMENT

- **Declarative Approach**
  - You specify **what** the system should look like, rather than how to get there.
  - It is like ordering pizza ***not how*** to make the pizza.
- **Idempotency**
  - Ansible playbooks can be executed multiple times without changing the system unnecessarily.
  - If a task's desired state is already achieved, Ansible skips that task.
- **Agentless Architecture**
  - Unlike Puppet or Chef, Ansible does not require an agent on managed nodes.
  - Uses **SSH** (or WinRM for Windows) to execute tasks remotely.
  - For example, Uber reaches the drivers and riders without any agent
- **Infrastructure as Code (IaC)**
  - Ansible configurations are stored as YAML files
  - Instead of manually installing httpd you write a script, and it does it for you.
  - Playbooks define system states using **tasks**, **roles**, and **modules**.
- **Version Control & Reusability**
  - Configuration files (playbooks, roles) can be stored in **Git**, making it easy to track changes and rollback if needed.

INTRODUCTION TO ANSIBLE MODULES

What are Modules?

In simple terms, **modules are like ready-made tools that help Ansible get things done easily and efficiently**!

Think of **Ansible modules** as **Lego blocks** that do specific tasks for you.

Real-Life Example:

Imagine you are assembling furniture. Instead of using just your hands, you have tools like:

- 🔧 **Screwdriver (yum module)** – to tighten screws (install Apache – httpd)
- 🔨 **Hammer** – to drive nails => (this is like a module in Ansible)
- 📏 **Measuring tape** – to check dimensions => (this is like a module in Ansible)

Each of these tools does a **specific** job to help you build furniture more easily.

Ansible **modules** are like these tools for IT tasks.

For example, if you need to:

1. **Install software** → Use the **yum** or **dnf** module
2. **Copy files** → Use the **copy module**
3. **Create a user** → Use the **user module**
4. **Restart a service** → Use the **service module**

$ansible-doc -l

Example in Action:

Instead of logging into a server and typing installation commands manually, you can use an Ansible module like this: ***dnf is the module*** (like your screwdriver) => It installs the **httpd** package (Apache Web Server)

Mastering Ansible Configuration File

A major part of the Control Node is the Ansible Configuration file and the default hosts files:

- **/etc/ansible/ansible.cfg**
- /etc/ansible/hosts (ansible looks for this file for all the target hosts/inventory/managed nodes.

Our discussion is to learn and master **"ansible.cfg"** file. Here are some very important parameters in that file that one should understand:

NOTE: If an "ansible.cfg" file is missing there are 2-ways to recover it:

(1) ….

(2) Copy and Paste the contents below into your ansible.cfg file:

[defaults]

 # inventory = /etc/ansible/hosts => Specifies the default inventory file location which contains the list of hosts Ansible will manage

library = /usr/share/my_modules/

module_utils = /usr/share/my_module_utils/

 # remote_tmp = ~/.ansible/tmp => Directory on the remote machine where Ansible temporarily stores files before executing them.

local_tmp = ~/.ansible/tmp

plugin_filters_cfg = /etc/ansible/plugin_filters.yml

forks = 5 => Maximum number of parallel processes – concurrency level

poll_interval = 15

 # sudo_user = root => Specifies the user to become when using privilege escalation

 # ask_sudo_pass = True => Prompts the user for the sudo password when running playbooks requiring privilege escalation

ask_pass = False => Prompts for SSH password if key-based authentication is not used.

remote_port = 22

host_key_checking = False

timeout = 10

```
# log_path        = /var/log/ansible.log
# private_role_vars = yes

# Uncomment this to disable SSH key host checking
# host_key_checking = False

# Uncomment to enable callbacks
# stdout_callback = default
# enable_task_debugger = False

[privilege_escalation]
# become=True
# become_method=sudo
# become_user=root
# become_ask_pass=False

[paramiko_connection]
# record_host_keys=False
# host_key_auto_add=True

[ssh_connection]
# ssh_args = -o ControlMaster=auto -o ControlPersist=60s
# control_path = %(directory)s/%%h-%%r
# pipelining = False
# scp_if_ssh = smart
```

INVENTORY

When Ansible commands are executed, Ansible looks for hosts (the managed nodes/target servers) in the following ways:

- Default Location: /etc/ansible/hosts [example: ansible all –m ping]
- Custom Location: /home/ansadmin/**hosts** [example: ansible all –m ping **–i hosts**]

The above two file paths can be updated in this file:

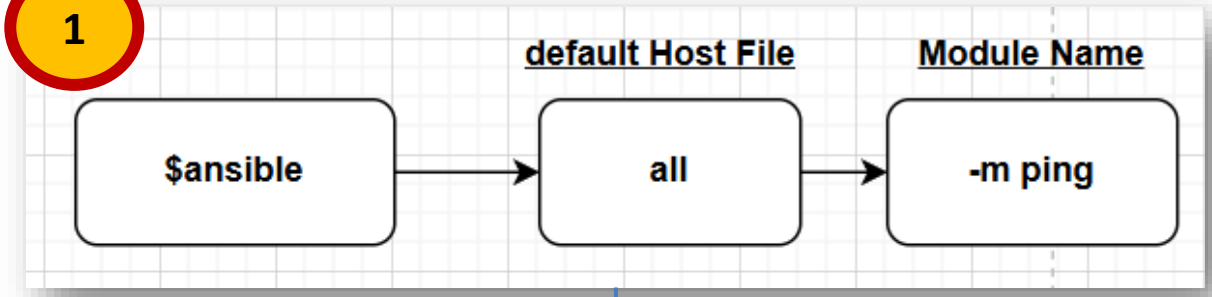
- /etc/ansible/ansible.cfg (default)

https://docs.ansible.com/ansible/9/collections/ansible/builtin/yum_module.html

ANSIBLE AD-HOC COMMAND INVENTORY

Inventory file is a collection of hosts(nodes) which are managed by ansible control node.

1



"all" = **Target Servers**

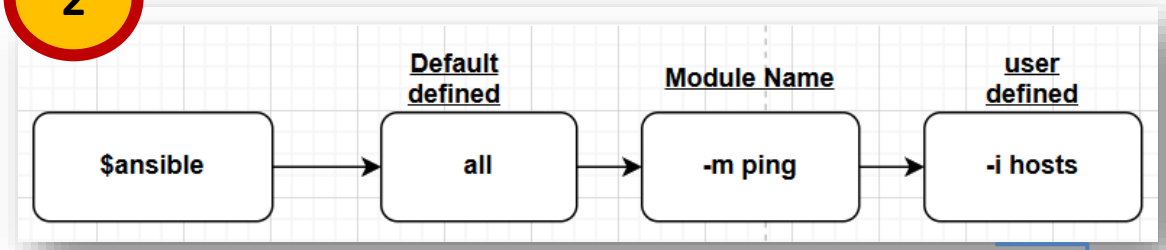
all means **all groups**: **[web_servers]** and **[app_servers]**

$cat /etc/ansible/hosts

```
[web_servers]
server1.example.com
server2.example.com

[app_servers]
app1.example.com
```

2



3

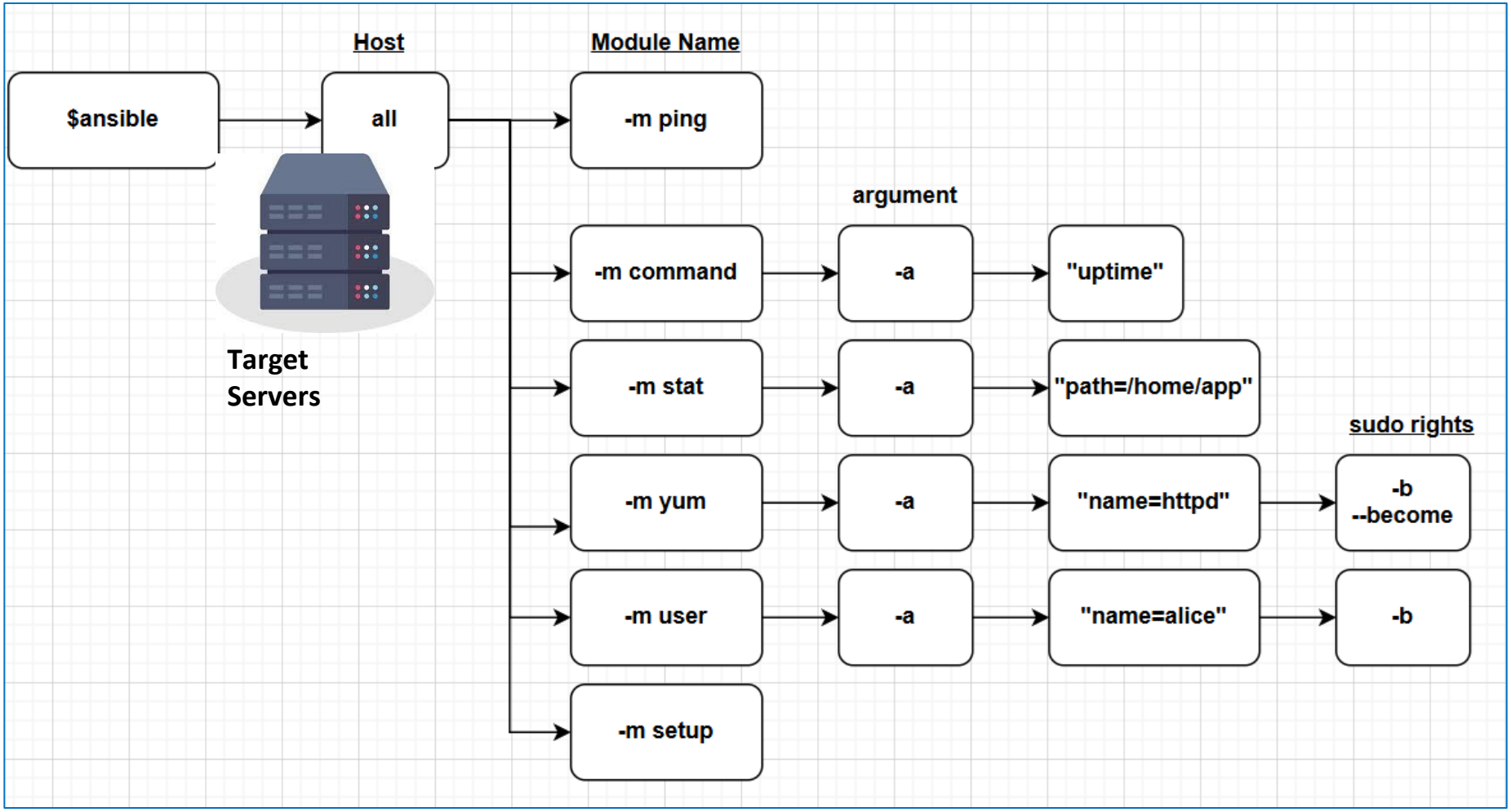
Ansible Configuration File_ default inventory

#more /etc/ansible/ansible.cfg

cat /home/ansadmin/hosts

```
[rhel]
db1.example.com
```

ANSIBLE AD-HOC COMMAND DETAIL



https://docs.ansible.com/ansible-core/devel/getting_started/basic_concepts.html
https://docs.ansible.com/ansible-core/devel/inventory_guide/index.html
https://docs.ansible.com/ansible-core/devel/reference_appendices/config.html#ansible-configuration-settings

ANSIBLE PLAYBOOK (BASICS)

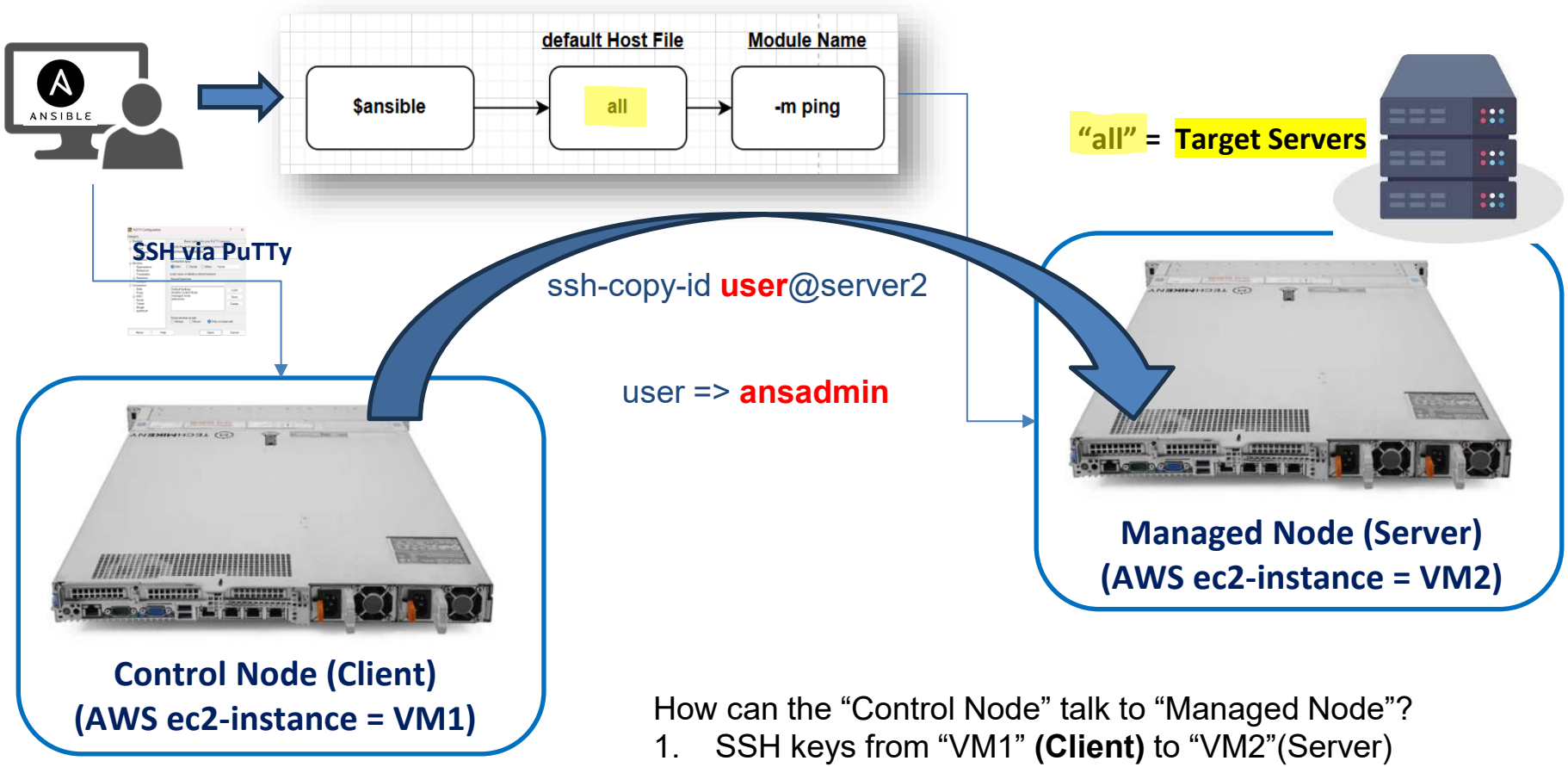
- A playbook is a text file written in **YAML** (YAML Ain't Markup Language) format and is normally saved as .yml => for example, **"create_user.yml"**
- The playbook begins with a line consisting of **three dashes** (**---**) as a start of document marker.
- An item in a YAML list starts with a **single dash** followed by a space.
- hosts and tasks are mandatory items in a playbook
- The playbook primarily uses indentation with space characters to indicate the structure of its data
- Modules are used to perform tasks
- Comment start with #

```
---
- hosts: all
  become: true
  tasks:
  - user: name=john
```

Ad-hoc command:
ansible all –m user –a "name=john" –b

ANSIBLE PLAYBOOK
AWS LAB

1. Open an Account in AWS
2. Setup EC2 Instance
3. Setup a hostname
4. Create an ansible user called “ansadmin”
5. Add the user to sudoers file giving“ansadmin” root privileges.
6. Generate SSH public/private keys
7. Enable password-based Login
8. Install Ansible



How can the "Control Node" talk to "Managed Node"?

1. SSH keys from "VM1" **(Client)** to "VM2"(Server)

ssh-copy-id user@server2

This command does the following:

•Appends server1's public key into server2:/home/user/.ssh/**authorized_keys**

Now server2 trusts server1 to log in **as that user** using key authentication.